

Event Loop :

The event loop is an important concept in JavaScript that enables asynchronous programming by handling tasks efficiently. Since JavaScript is single-threaded, it uses the event loop to manage the execution of multiple tasks without blocking the main thread.

The Event Loop manages the asynchronous execution process in JavaScript by continuously monitoring the call stack. When the call stack becomes empty, it moves callbacks from the microtask and macrotask queues into the call stack for execution.

Example :

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Explanation:

1. A and C are synchronous → executed first.
2. setTimeout is asynchronous → its callback moves to the macrotask queue.
3. The Event Loop runs it after all synchronous code finishes.

Working of Event Loop

1. JS Engine runs code line-by-line in the Call Stack.
2. If it finds asynchronous code (like setTimeout), that task is sent to Web APIs.
3. After the async operation finishes, its callback moves to either:
 - Microtask Queue → for Promises, or
 - Callback (Macrotask) Queue → for timers, events.
4. The Event Loop constantly checks:
 - If the Call Stack is empty, it takes tasks from queues.
 - It always clears the Microtask Queue first, then moves to the Macrotask Queue.
5. The process repeats continuously.

Example : Microtask and Macrotask

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

Explanation :

1. Call Stack Execution (Synchronous Code)

- JavaScript starts by running code line by line in the Call Stack.

Line	Action	Queue	Priority
console.log("Start")	Prints Start	—	High (synchronous)
setTimeout(...)	Registers callback in Web API (0ms timer)	Goes to Macrotask Queue after 0ms	Low
Promise.resolve().then(...)	Promise resolved immediately, callback sent to Microtask Queue	Microtask Queue	High
console.log("End")	Prints End	—	High (synchronous)

2. Queue Processing (After Stack Empty)

When the main (synchronous) code is done, the Call Stack becomes empty.

Now the Event Loop checks:

1. Are there any Microtasks? → Yes (Promise.then)
Execute all microtasks first.
2. Then check Macrotasks (like setTimeout) → Execute next.

Callbacks :

A callback is simply a function passed as an argument to another function, which is then executed later — either after some operation completes or at a specific time

Why Are Callbacks Used?

1. Handling Asynchronous Tasks:

Since JavaScript operates on a single thread, it can't perform multiple actions at once. To avoid blocking the program while waiting for time-consuming operations — like server requests, file reading, or timers — JavaScript uses callbacks. These functions are executed only after the long-running task has finished.

2. Providing Flexibility and Control:

Callbacks allow developers to define custom behavior after a process completes. This means a built-in function or library can perform its own task while still letting you decide what should happen next, without needing to modify its internal logic.

Types of Callbacks in JavaScript

Type	Description	Example
Synchronous Callback	Runs immediately within the main function	array.forEach(), map()
Asynchronous Callback	Runs later, after async task	setTimeout(), fetch()
Error-First Callback	Asynchronous callback with error parameter (Node.js style)	fs.readFile((err, data) => {...})
Event Callback	Executes in response to an event (user or system)	addEventListener("click", callback)

Example 1 : Synchronous Callback

```
function calculate(a, b, callback) {  
  let result = a + b;  
  callback(result); // executed right away  
}  
  
function display(sum) {  
  console.log("The sum is:", sum);  
}  
  
calculate(10, 20, display);
```

Example 2 : Asynchronous Callback

```
console.log("Start");  
  
setTimeout(() => {  
  console.log("Data loaded!");  
}, 2000);  
  
console.log("End");
```

Example 3 : Event Callback

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Callback in Event Handling</title>  
</head>
```

```
<body>

<h2>Callback in Event Handling Example</h2>
<button id="clickMe">Click Me!</button>

<script>
  // Attaching an event listener with an anonymous callback
  document.getElementById("clickMe").addEventListener("click", function() {
    console.log("Button was clicked!");
    alert("You just clicked the button!");
  });
</script>

</body>
</html>
```

Promises

A Promise in JavaScript is an object that represents the eventual completion (or failure) of an asynchronous operation and its resulting value.

It's essentially a placeholder for a value that is currently unknown but will be available in the future. Promises offer a cleaner, more structured alternative to managing asynchronous code compared to using deeply nested callbacks (often called "Callback Hell").

Promise States

A Promise can be in one of three states:

State	Meaning
Pending	The initial state — operation not yet completed.
Fulfilled	The operation completed successfully — returns a value.
Rejected	The operation failed returns an error reason.

Consuming a Promise

You interact with the result of a promise using two main methods, which are chained onto the promise object:

Method	Role	State Handled
.then()	Registers a callback function to run when the promise is successfully Fulfilled.	Fulfilled
.catch()	Registers a callback function to run when the promise is Rejected (i.e., an error occurred).	Rejected
.finally()	Registers a callback to run regardless of whether the promise was Fulfilled or Rejected. Used for cleanup tasks.	Settled

Example :

```
const myPromise = new Promise((resolve, reject) => {
  let success = true;

  if (success) {
    resolve("Task completed successfully!");
  } else {
    reject("Something went wrong!");
  }
})
```

```
});

myPromise
  .then(result => console.log(result)) // runs if resolved
  .catch(error => console.log(error)) // runs if rejected
  .finally(() => console.log("Promise finished."));
```

Example2 :

```
console.log("Start");

const fetchData = new Promise((resolve, reject) => {
  setTimeout(() => {
    const success = Math.random() > 0.5; // Random true/false

    if (success) {
      resolve("Data loaded successfully!");
    } else {
      reject("Failed to load data. Please try again.");
    }
  }, 2000);
});

fetchData
  .then(response => console.log(response))
  .catch(error => console.error(error))
  .finally(() => console.log("Operation complete."));

console.log("End");
```

Example 3 : Fetching data through API using Promises

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Random User Fetch Example</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin-top: 50px;
      background-color: #f5f5f5;
    }

    .user-card {
      background: white;
      border-radius: 10px;
      padding: 20px;
      width: 300px;
      margin: 20px auto;
      box-shadow: 0 0 10px rgba(0,0,0,0.1);
    }

    img {
      border-radius: 50%;
      margin-bottom: 10px;
    }

    button {
      background-color: #007bff;
      color: white;
      border: none;
      padding: 10px 15px;
      border-radius: 5px;
      cursor: pointer;
    }

    button:hover {
      background-color: #0056b3;
    }
  </style>
</head>
<body>

  <h2>&img alt="GitHub logo" data-bbox="185 865 205 880"/> Random User Generator</h2>
  <button id="fetchBtn">Get Random User</button>

```



```

<div id="userContainer" class="user-card" style="display: none;">
  <img id="userImage" src="" alt="User Image" width="100">
  <h3 id="userName"></h3>
  <p id="userEmail"></p>
</div>

<script>
  const fetchBtn = document.getElementById("fetchBtn");
  const userContainer = document.getElementById("userContainer");
  const userName = document.getElementById("userName");
  const userEmail = document.getElementById("userEmail");
  const userImage = document.getElementById("userImage");

  fetchBtn.addEventListener("click", () => {
    console.log("Fetching random user...");

    fetch("https://randomuser.me/api/")
      .then(response => response.json())
      .then(data => {
        const user = data.results[0];
        console.log("Random User:", user);

        userImage.src = user.picture.large;
        userName.textContent = `${user.name.first} ${user.name.last}`;
        userEmail.textContent = user.email;

        userContainer.style.display = "block";
      })
      .catch(error => console.log("Error fetching data:", error))
      .finally(() => console.log("Request completed."));
  });
</script>

</body>
</html>

```

Callback Hell:

Callback Hell happens when multiple nested callbacks are used to handle asynchronous operations one after another creating deeply indented, hard-to-read, and hard-to-maintain code.

Reason of Callback Hell :

JavaScript is single-threaded and handles asynchronous tasks (like reading files, making API calls, or setting timers) using callbacks.

When one async task depends on another, developers sometimes keep nesting callbacks — and that's how callback hell begins.

Example of Callback Hell

```
console.log("Start Order Process");

function takeOrder(callback) {
  setTimeout(() => {
    console.log("Step 1: Order received from customer");
    callback();
  }, 1000);
}

function prepareFood(callback) {
  setTimeout(() => {
    console.log("Step 2: Preparing food");
    callback();
  }, 1000);
}

function packFood(callback) {
  setTimeout(() => {
    console.log("Step 3: Packing the food");
    callback();
  }, 1000);
}

function deliverFood(callback) {
  setTimeout(() => {
    console.log("Step 4: Delivering food to customer");
    callback();
  }, 1000);
}

function confirmDelivery() {
  setTimeout(() => {
    console.log("Step 5: Order delivered successfully!");
  }, 1000);
}
```

```
}

// Nested callbacks -- Callback Hell
takeOrder(() => {
  prepareFood(() => {
    packFood(() => {
      deliverFood(() => {
        confirmDelivery();
      });
    });
  });
});

console.log("End Order Process");
```

Two ways to avoid Callback Hell :

- 1 Promises
- 2 async & await

Example of avoiding Callback Hell using Promises:

Promise Chaining means linking multiple `.then()` methods together, where the output of one `.then()` becomes the input for the next one.

Each `.then()` returns a new Promise, allowing you to perform a sequence of asynchronous operations in a clean, readable order — without falling into “callback hell.”

Why Promise Chaining Is Necessary

Without chaining, handling multiple asynchronous tasks (like fetching data, processing it, then saving it) would require nested callbacks, which quickly becomes messy and hard to debug.

Promise chaining:

1. Keeps code flat and readable.
2. Ensures each step waits for the previous one to finish.
3. Makes error handling easier, since one `.catch()` can handle all rejections in the chain.

```
console.log("Start Order Process");

// Each function returns a Promise
function takeOrder() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 1: Order received from customer");
      resolve();
    }, 1000);
  });
}

function prepareFood() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 2: Preparing food");
      resolve();
    }, 1000);
  });
}

function packFood() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 3: Packing the food");
      resolve();
    }, 1000);
  });
}

function deliverFood() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 4: Delivering food to customer");
      resolve();
    }, 1000);
  });
}

function confirmDelivery() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 5: Order delivered successfully!");
      resolve();
    }, 1000);
  });
}
```

```

    });
}

// Promise Chaining -- clean and readable
takeOrder()
    .then(prepareFood)
    .then(packFood)
    .then(deliverFood)
    .then(confirmDelivery)
    .then(() => console.log(" All steps completed!"))
    .catch((error) => console.log("Error:", error));

console.log("End Order Process");

```

Async/Await

async and await are modern JavaScript keywords introduced in ES2017 (ES8) that make working with Promises simpler and more readable.

They allow you to write asynchronous code that looks and behaves like synchronous code, making it easier to understand and maintain.

Note : The await keyword can only be used inside a function that is declared with async.

Keyword	Definition
async	Placed before a function declaration (async function() {}). This keyword tells JavaScript that the function is asynchronous and will always return a Promise. Even if the function returns a simple value, it's implicitly wrapped in a resolved Promise.

await	Placed before a Promise call inside an async function. The await keyword pauses the execution of the async function until the Promise settles (resolves or rejects). Once settled, it returns the resolved value of the Promise.
-------	--

Benefits of Using Async/Await

1. It removes callback hell and promise chaining complexity.
2. It makes asynchronous logic linear and readable.
3. It allows use of try...catch for cleaner error handling.

Example :

```
console.log("Fetching random user data...");

async function getRandomUser() {
  try {
    const response = await fetch("https://randomuser.me/api/");
    const data = await response.json();
    const user = data.results[0];

    console.log("Name:", `${user.name.first} ${user.name.last}`);
    console.log("Email:", user.email);
    console.log("Country:", user.location.country);
  } catch (error) {
    console.log("Error fetching data:", error);
  } finally {
    console.log("Request completed.");
  }
}

getRandomUser();
```

Example of avoiding callback hell using async/await

```
console.log("Start Order Process");

function takeOrder() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 1: Order received from customer");
      resolve();
    }, 1000);
  });
}

function prepareFood() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 2: Preparing food");
      resolve();
    }, 1000);
  });
}

function packFood() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 3: Packing the food");
      resolve();
    }, 1000);
  });
}

function deliverFood() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Step 4: Delivering food to customer");
      resolve();
    }, 1000);
  });
}

function confirmDelivery() {
  return new Promise((resolve) => {
    setTimeout(() => {
```

```
        console.log("Step 5: Order delivered successfully!");
        resolve();
    }, 1000);
});
}
```

```
async function processOrder() {
    try {
        await takeOrder();
        await prepareFood();
        await packFood();
        await deliverFood();
        await confirmDelivery();
        console.log(" All steps completed!");
    } catch (error) {
        console.log("Error:", error);
    }
}
```

```
processOrder();
console.log("End Order Process");
```